

SPM - Cheatsheet Cluster & Esecuzione Moduli (orale)

Tutto ciò che serve per accedere al cluster ed eseguire **a mano** ogni esperimento, modulo per modulo. Copia-incolla diretto. I parametri sono quelli reali degli script di benchmark.

Hardware. node01-node08: 2x Intel Xeon E5-2640 v2 (Ivy Bridge), 16 core fisici / 32 logici (HT), 2 NUMA, **no AVX2**. node09: AMD EPYC 7301 (Zen 1) + GPU **NVIDIA A30**. Login node **spmln**: Xeon E5-2650 v3 (Haswell), **ha AVX2**. **Regola d'oro:** compila **sul nodo su cui esegui** (per `-march=native`); per M4 usa `-march=ivybridge` (vedi sotto).

salloc NON sposta la shell sul compute node. Verificato: dopo `salloc` la shell resta su `spmln` (login), l'allocazione e' solo riservata; ci si arriva **solo con srun**. Conseguenza pratica: `make/g++` lanciati dopo `salloc` compilano **sul login (Haswell, AVX2)**, e il binario su Ivy Bridge muore con `Illegal instruction (core dumped)`. Per M1/M2/M3 le build vanno quindi passate a `srun` (`srun make ...`, `srun g++ ...`); M4 e' l'unico che si compila dal login, perche' il suo target `cluster` forza `-march=ivybridge`. Corollario: `nvcc` **non esiste sul login**, sta solo su node09 -> anche la build CUDA va via `srun`.

0. Accesso, trasferimento file, SLURM

```
# login (loginname = parte locale della mail UNIPI, minuscolo). Da fuori UNIPI: prima la VPN "Connect Tunne
ssh loginname@spmcluster.unipi.it
```

```
# trasferire una cartella modulo sul cluster (ATTENZIONE alla / finale; NON copiare dentro ~/ diretto: romp
# NB: sul cluster i moduli stanno in ~/module_1 .. ~/module_4 (non in ~/spm/)
```

```
rsync -rv module_3/ loginname@spmcluster.unipi.it:~/module_3/
```

```
# se per errore hai rotto i permessi della home: chmod 700 ~
```

```
# SLURM essenziale
```

<code>sinfo</code>	<i># partizioni e nodi disponibili (verifica i nomi: normal, gpu-excl, ...)</i>
<code>squeue -u \$USER</code>	<i># i miei job</i>
<code>scancel <jobid></code>	<i># uccidere un job</i>
<code>lscpu</code>	<i># topologia CPU del nodo (socket, core, thread)</i>
<code>numactl --hardware</code>	<i># nodi NUMA e distanze</i>

salloc vs srun vs sbatch - `salloc <flag>` -> riserva risorse e apre una shell che **tiene** l'allocazione: compili e lanci a mano. **È la modalità giusta per l'orale**. La shell resta sul **login node** (`hostname -> spmln`): l'allocazione e' riservata ma il compute node si tocca solo via `srun`. - `srun <flag> ./bin ->` lancia il programma **dentro** l'allocazione. Per MPI: `srun --mpi=pmix`. - `sbatch script.sh ->` sottomette lo script batch (i benchmark completi). Non serve all'orale.

Flag ricorrenti: `--nodes=N` (nodi), `--ntasks=R` (processi/rank), `--ntasks-per-node`, `--cpus-per-task=T` (core per task -> OpenMP), `--exclusive` (nodo tutto per sé, timing pulito), `--partition=normal` (o `gpu-excl` per node09), `--time=HH:MM:SS`.

1. Modulo 1 - SIMD (gira su node09, partizione gpu-excl)

CLI: `./bin/<binario> N P [seed] [key_space] [reps]` - binari: `plain_baseline`, `plain_autovec`, `avx2`, `cuda_kernel`.

Allocazione + build

```
cd ~/module_1
salloc --partition=gpu-excl --nodelist=node09 --ntasks=1 --cpus-per-task=1 --exclusive --time=00:25:00
srun make all      # plain_baseline, plain_autovec, avx2, dataset_creator (srun: compila SU node09)
```

Demo minima (la cosa che il prof probabilmente chiede)

```
srun ./bin/plain_baseline 100000000 256 42 0 11
srun ./bin/plain_autovec 100000000 256 42 0 11
srun ./bin/avx2 100000000 256 42 0 11
make test_correctness # N=16 P=4: confronto checksum FNV-1a (OK/FAIL)
```

Esperimento 1 - sweep N (P=256, seed=42, reps=11)

```
for N in 1000000 10000000 50000000 100000000 200000000; do
  echo "--- N=$N P=256 ---"
  srun ./bin/plain_baseline $N 256 42 0 11
  srun ./bin/plain_autovec $N 256 42 0 11
  srun ./bin/avx2 $N 256 42 0 11
done
```

Esperimento 2 - sweep P (N=100M)

```
for P in 2 4 8 16 32 64 128 256 512 1024; do
  echo "--- N=100M P=$P ---"
  srun ./bin/avx2 100000000 $P 42 0 11
done
```

Esperimento 3 - sweep key_space (N=100M, P=256) -> effetto duplicati

```
for KS in 0 1000 100000 1000000 1000000000; do
  echo "--- key_space=$KS ---"
  srun ./bin/avx2 100000000 256 42 $KS 11
done
```

CUDA (solo node09)

```
# nvcc NON e' sul login: la build va passata a srun (node09). Il LD_LIBRARY_PATH serve invece
# nella shell, perche' srun lo propaga al nodo dove gira il binario.
export LD_LIBRARY_PATH=/usr/local/cuda-12.3/lib64:$LD_LIBRARY_PATH
srun bash -c 'export PATH=/usr/local/cuda-12.3/bin:$PATH; nvcc -std=c++17 -O3 -I include -gencode arch=compute_750 -code=sm_75'
srun ./bin/cuda_kernel 100000000 256 42 0 11 # stampa breakdown H2D / kernel / D2H
for N in 1000000 10000000 50000000 100000000 200000000; do srun ./bin/cuda_kernel $N 256 42 0 11; done
```

Cosa far notare: throughput piatto su N/P/key_space => bandwidth-bound (I~0.33). Autovec ~1.43x, AVX2 ~1.30x (l'autovec batte le intrinsics). CUDA: kernel 808 GB/s ma end-to-end ~1.12x (PCIe domina).

2. Modulo 2 - C++ threads (1 nodo, partizione normal)

CLI: -nr <NR> -ns <NS> -seed <S> -max-key <K> -p <P> [-t <T>] - binari: hashjoin_seq, hashjoin_par.

Allocazione + build (via srun, così -march=native = Ivy Bridge)

```
cd ~/module_2
salloc --nodes=1 --ntasks=1 --cpus-per-task=32 --exclusive --time=00:30:00
# senza il srun davanti compili sul login (Haswell) -> SIGILL all'esecuzione
srun g++ -O3 -std=c++20 -Wall -Wextra -march=native -pthread -Iinclude src/hashjoin_seq.cpp -o hashjoin_seq
srun g++ -O3 -std=c++20 -Wall -Wextra -march=native -pthread -Iinclude src/hashjoin_parallel.cpp -o hashjoin_parallel
```

Demo minima

```
srunk --cpus-per-task=32 ./hashjoin_seq -nr 10000000 -ns 20000000 -seed 42 -max-key 1000000 -p 128
srunk --cpus-per-task=32 ./hashjoin_par -nr 10000000 -ns 20000000 -seed 42 -max-key 1000000 -p 128 -t 16
# correttezza naive (NR, NS <= 500):
./hashjoin_par -nr 50 -ns 80 -seed 13 -max-key 8 -p 4 -t 4      # -> naive_verify=PASS
```

Strong scaling (NR in {10M, 20M}, NS=2*NR, seed=42, max-key=1M, P=128)

```
for NR in 10000000 20000000; do NS=$((NR*2))
  srunk --cpus-per-task=32 ./hashjoin_seq -nr $NR -ns $NS -seed 42 -max-key 1000000 -p 128
  for T in 1 2 4 8 12 16 20 24 28 32; do
    srunk --cpus-per-task=32 ./hashjoin_par -nr $NR -ns $NS -seed 42 -max-key 1000000 -p 128 -t $T
  done
done
```

Weak scaling (NR = 1MT, NS = 2NR)

```
for T in 1 2 4 8 16 20 24 32; do WNR=$((1000000*T)); WNS=$((WNR*2))
  srunk --cpus-per-task=32 ./hashjoin_par -nr $WNR -ns $WNS -seed 42 -max-key 1000000 -p 128 -t $T
done
```

Sensibilità a P (NR=10M, T=20) - mostra l'ottimo a P=512

```
for P in 16 32 64 128 256 512 1024 2048 4096; do
  srunk --cpus-per-task=32 ./hashjoin_par -nr 10000000 -ns 20000000 -seed 42 -max-key 1000000 -p $P -t 20
done
```

Densità di duplicati (NR=10M, P=128, T=20) - vario max-key

```
for MK in 100 10000 100000 1000000 10000000; do
  srunk --cpus-per-task=32 ./hashjoin_par -nr 10000000 -ns 20000000 -seed 42 -max-key $MK -p 128 -t 20
done
```

Punto cross-module (confronto diretto con M3/M4)

```
srunk --cpus-per-task=32 ./hashjoin_par -nr 50000000 -ns 100000000 -seed 42 -max-key 25000000 -p 256 -t 32
```

Cosa far notare: breakdown su stderr (scatter ~53%, join, histogram). Dip NUMA a p=20. Amdahl f~0.078. A P=512 entra in L3 -> 11.4x.

3. Modulo 3 - OpenMP (1 nodo, normal). Affinity OBBLIGATORIA.

CLI: -nr -ns -seed -max-key -p -mode loop|task -t [-skew <rho> -hot <H>] - binari: hashjoin_seq, hashjoin_omp, hashjoin_omp_runtime (per lo schedule-sweep). Env sempre: OMP_PROC_BIND=close OMP_PLACES=cores.

Allocazione + build

```
cd ~/module_3
salloc --nodes=1 --ntasks=1 --cpus-per-task=32 --exclusive --time=00:20:00 --partition=normal
# il target usa -march=native hardcoded: senza srunk compila sul login (Haswell) -> SIGILL
srunk make cluster CXX=g++      # hashjoin_seq + hashjoin_omp con -march=native
srunk make cluster_runtime CXX=g++ # hashjoin_omp_runtime (schedule(runtime)) con -march=native
export OMP_PROC_BIND=close OMP_PLACES=cores
```

Demo minima

```
OMP_NUM_THREADS=16 srun --cpus-per-task=32 ./hashjoin_omp -nr 10000000 -ns 20000000 -seed 42 -max-key 5000000
OMP_NUM_THREADS=8 srun --cpus-per-task=32 ./hashjoin_omp -nr 10000000 -ns 20000000 -seed 42 -max-key 5000000
./hashjoin_seq -nr 10000000 -ns 20000000 -seed 42 -max-key 5000000 -p 128
```

Strong scaling (NR=10M, NS=20M, max-key=5M, P=128) - 4 combinazioni

```
for MODE in loop task; do
  for WL in "uniform" "skewed -skew 0.9 -hot 4"; do
    for T in 1 2 4 8 16 20 32; do
      set -- $WL
      [ "$1" = "uniform" ] && EXTRA="" || EXTRA="$WL"
      OMP_NUM_THREADS=$T srun --cpus-per-task=32 ./hashjoin_omp \
        -nr 10000000 -ns 20000000 -seed 42 -max-key 5000000 -p 128 -mode $MODE -t $T $EXTRA
    done
  done
done
```

Weak scaling (BASE_NR=2M per thread -> NR=2MT, NS=2NR)

```
for MODE in loop task; do for T in 1 2 4 8 16; do NR=$((2000000*T)); NS=$((NR*2))
  OMP_NUM_THREADS=$T srun --cpus-per-task=32 ./hashjoin_omp -nr $NR -ns $NS -seed 42 -max-key 5000000 -p 128
done; done
```

Schedule-sweep (binario runtime, T=8) - dynamic batte static su skew

```
for SCHED in static "static,1" "dynamic,1" "dynamic,4" "guided,1"; do
  echo "=== $SCHED (uniform) ==="
  OMP_NUM_THREADS=8 OMP_SCHEDULE="$SCHED" srun --cpus-per-task=32 ./hashjoin_omp_runtime -nr 10000000 -ns 20000000 -seed 42 -max-key 5000000 -p 128
  echo "=== $SCHED (skewed) ==="
  OMP_NUM_THREADS=8 OMP_SCHEDULE="$SCHED" srun --cpus-per-task=32 ./hashjoin_omp_runtime -nr 10000000 -ns 20000000 -seed 42 -max-key 5000000 -p 128
done
```

Validazione correttezza (NR=1M, NS=2M, max-key=500k, P=128)

```
for T in 1 2 4 8 16; do
  ./hashjoin_omp -nr 1000000 -ns 2000000 -seed 42 -max-key 500000 -p 128 -mode loop -t $T
  ./hashjoin_omp -nr 1000000 -ns 2000000 -seed 42 -max-key 500000 -p 128 -mode task -t $T
done
# confronta join_count/checksum1/checksum2 con hashjoin_seq
```

Cosa far notare: loop vince su uniforme, task su skew (LPT + split intra-partizione, rompe il tetto H/T).
Schedule: uniforme tutte ~uguali, skew dynamic -38%. M3 loop 11.46x vs M2 7.86x a T=16.

4. Modulo 4 - MPI + ibrido (multi-nodo, normal)

CLI: -nr -ns -seed -max-key -p [-t <T>] [-skew <rho> -hot <H>] - binari: hashjoin_mpi, hashjoin_mpi_omp, hashjoin_seq. **Parametri esperimenti:** NR=50M, NS=100M, seed=42, max-key=25M, P=256.

Build (CRITICO: -march=ivybridge, non native!)

```
cd ~/module_4
# NB: "module load openmpi5" NON esiste su questo cluster (Lmod: "unknown module").
# Non serve: mpicxx/mpirun sono gia' nel PATH di default e sono Open MPI 5.0.3.
```

```
which mpicxx # -> /opt/ohpc/pub/mpi/openmpi5-gnu12/5.0.3/bin/mpicxx
make cluster MPICXX=mpicxx CXX=g++ # compila mpi, ibrido, seq con -march=ivybridge -mtune=ivybridge
# unico modulo compilabile dal login, proprio perche' non usa -march=native
# Perche': il login e' Haswell (AVX2), i nodi di calcolo Ivy Bridge (no AVX2).
# -march=native sul login -> SIGILL sui nodi di calcolo.
```

Demo locale (sul frontend, senza SLURM)

```
mpirun -n 4 ./hashjoin_mpi -nr 1000000 -ns 2000000 -seed 42 -max-key 500000 -p 32
mpirun -n 2 ./hashjoin_mpi_omp -nr 1000000 -ns 2000000 -seed 42 -max-key 500000 -p 32 -t 4
```

Baseline sequenziale (nodo dedicato, 1 core)

```
salloc --partition=normal --nodes=1 --ntasks=1 --time=00:10:00 --exclusive
srun --nodes=1 --ntasks=1 --cpus-per-task=1 ./hashjoin_seq -nr 50000000 -ns 100000000 -seed 42 -max-key 250000000
# variante skewed: ... -skew 0.9 -hot 4
```

Strong scaling - MPI puro (ranks = nodi x 32)

```
for NODES in 1 2 4 8; do RANKS=$((NODES*32))
  salloc --partition=normal --nodes=$NODES --ntasks-per-node=32 --time=00:25:00 --exclusive \
  srun --nodes=$NODES --ntasks=$RANKS --ntasks-per-node=32 --mpi=pmix \
  ./hashjoin_mpi -nr 50000000 -ns 100000000 -seed 42 -max-key 250000000 -p 256
done
```

Strong scaling - ibrido (1 rank/nodo x 32 thread)

```
for NODES in 1 2 4 8; do
  salloc --partition=normal --nodes=$NODES --ntasks-per-node=1 --cpus-per-task=32 --time=00:25:00 --exclusive
  export OMP_NUM_THREADS=32 OMP_PROC_BIND=close OMP_PLACES=cores
  srun --nodes='$NODES' --ntasks='$NODES' --ntasks-per-node=1 --cpus-per-task=32 --mpi=pmix \
  ./hashjoin_mpi_omp -nr 50000000 -ns 100000000 -seed 42 -max-key 250000000 -p 256 -t 32'
done
```

Singolo punto a mano (es. il collasso a 128 rank = 4 nodi x 32)

```
salloc --partition=normal --nodes=4 --ntasks-per-node=32 --time=00:15:00 --exclusive
srun --nodes=4 --ntasks=128 --ntasks-per-node=32 --mpi=pmix ./hashjoin_mpi -nr 50000000 -ns 100000000 -seed 42 -max-key 250000000 -p 256
```

Weak scaling (per-rank fisso: 2M R / 4M S per rank)

```
# MPI puro: NR = 2M x (NODESx32)
for NODES in 1 2 4 8; do RANKS=$((NODES*32)); NR=$((2000000*RANKS)); NS=$((4000000*RANKS))
  salloc --nodes=$NODES --ntasks-per-node=32 --time=00:25:00 --exclusive \
  srun --nodes=$NODES --ntasks=$RANKS --ntasks-per-node=32 --mpi=pmix \
  ./hashjoin_mpi -nr $NR -ns $NS -seed 42 -max-key 250000000 -p 256
done

# Ibrido: NR = 2M x NODES (1 rank/nodo)
for NODES in 1 2 4 8; do NR=$((2000000*NODES)); NS=$((4000000*NODES))
  salloc --nodes=$NODES --ntasks-per-node=1 --cpus-per-task=32 --time=00:25:00 --exclusive bash -c '
  export OMP_NUM_THREADS=32 OMP_PROC_BIND=close OMP_PLACES=cores
  srun --nodes='$NODES' --ntasks='$NODES' --ntasks-per-node=1 --cpus-per-task=32 --mpi=pmix \
  ./hashjoin_mpi_omp -nr '$NR' -ns '$NS' -seed 42 -max-key 250000000 -p 256 -t 32'
done
```

Breakdown per fase (4 nodi) - uniforme e skewed, puro e ibrido

```
salloc --partition=normal --nodes=4 --ntasks-per-node=32 --time=00:15:00 --exclusive
# puro uniforme:
srun --nodes=4 --ntasks=128 --ntasks-per-node=32 --mpi=pmix ./hashjoin_mpi -nr 50000000 -ns 100000000 -seed
# puro skewed:
srun --nodes=4 --ntasks=128 --ntasks-per-node=32 --mpi=pmix ./hashjoin_mpi -nr 50000000 -ns 100000000 -seed
# ibrido uniforme:
export OMP_NUM_THREADS=32 OMP_PROC_BIND=close OMP_PLACES=cores
srun --nodes=4 --ntasks=4 --ntasks-per-node=1 --cpus-per-task=32 --mpi=pmix ./hashjoin_mpi_omp -nr 50000000
# ibrido skewed: aggiungi -skew 0.9 -hot 4
```

Correttezza (input piccolo, naive $O(N^2)$)

```
mpirun -n 4 ./hashjoin_mpi -nr 200 -ns 200 -seed 1 -max-key 1000 -p 16 # -> join_count / naive_verify
```

Cosa far notare: ibrido 2.8x->12.0x (1->8 nodi); MPI puro collassa a 4 nodi/128 rank (Alltoallv erratico).
Weak: ibrido 0.58 vs puro 0.21 (Hockney: start-up prop. a rank). 8 fasi cronometrate. Su skew M3 (1 nodo) batte M4.

4-bis. Terminali pronti per l'orale (un tab per modulo)

Sul Mac, da SPM/oral_terminals/:

```
./open_terminals.sh # 4 tab: M1, M2, M3, M4 (Cmd+1..4 per spostarsi)
./open_terminals.sh 3 # solo il modulo 3
```

Ogni tab apre l'ssh al cluster con la **history del modulo già caricata**: freccia su per scorrere gli esperimenti, !n per eseguire il comando n del menu, spm per rimostrare il menu, Ctrl+R per cercare. La history e' agganciata a SPM_MOD, esportata: si ricarica identica anche nella shell aperta da salloc e sui compute node via srun.

Installato sul cluster (nessun file esistente modificato: ~/.bashrc già sorgia ~/.bashrc.d/*): ~/.bashrc.d/spm_oral.sh, ~/.spm_hist_m1..m4, ~/.spm_menu_m1..m4. Per rigenerare dopo aver modificato gli spec: ./gen.sh && ./deploy.sh.

5. Se qualcosa va storto

- **sbatch/srun dice partizione inesistente** -> sinfo per il nome esatto. Partizioni reali: normal (node01-08, limite 30 min), gpu-excl (node09, 30 min), gpu-shared (node09, 10 min).
 - **SIGILL / "Illegal instruction (core dumped)" (M1/M2/M3)** -> hai compilato -march=native sul login (Haswell/AVX2) perche' salloc non ti sposta sul compute node. Ricompila mettendo srun davanti: srun make cluster CXX=g++, srun g++
 - **SIGILL (M4)** -> ricompila make cluster (-march=ivybridge), non -march=native.
 - **nvcc: command not found (M1)** -> nvcc sta solo su node09: la build CUDA va lanciata con srun, non dalla shell del login.
 - **module load openmpi5** -> "unknown module" -> normale, quel modulo non esiste qui. mpicxx e' gia' Open MPI 5.0.3 nel PATH: verifica con which mpicxx.
 - **SSH rifiuta il login dopo un rsync** -> permessi home: chmod 700 ~.
 - **OpenMP non usa i thread giusti** -> setta OMP_NUM_THREADS e -t allo stesso valore; verifica OMP_PROC_BIND=close OMP_PLACES=cores.
 - **MPI non parte sotto srun** -> manca --mpi=pmix (e module load openmpi5).
 - **Non ricordi i flag di un binario** -> leggi usage nel sorgente o lancia con argomenti errati per stampare l'help; non inventare flag.
-

6. Riferimento flag (dalla teoria) - cosa fanno

6.1 Compilatore C++ (g++) - ottimizzazione e ISA

Flag	Cosa fa	Dove nel progetto
-O3	Massima ottimizzazione: inlining, auto-vettorizzazione, unrolling	tutti
-O2	Ottimizzazione senza alcune trasformazioni aggressive di -O3	-
-march=native	Genera istruzioni per la microarch del nodo di compilazione (abilita AVX2/FMA se presenti)	M1, M2, M3 (compila sul nodo)
-march=ivybridge -mtune=ivybridge	Fissa la ISA baseline (Ivy Bridge, no AVX2); evita SIGILL sui nodi di calcolo	M4 (login Haswell != nodi)
-mavx2	Abilita le istruzioni vettoriali AVX2 (registri 256-bit, <code>vpmulld</code> , ...)	M1
-mfma	Abilita Fused Multiply-Add (<code>a*b+c</code> in una istruzione)	M1
-std=c++20	Standard C++20 (serve per <code>std::barrier</code> , <code>std::jthread</code> , concepts)	tutti
-pthread	Compila/linka con i POSIX threads (sotto <code>std::thread</code>)	M2
-fopenmp	Abilita OpenMP: traduce le <code>#pragma omp</code> e linka il runtime	M3, M4 ibrido
-linclude	Aggiunge <code>include/</code> ai path degli header	tutti
-Wall -Wextra -Wpedantic	Abilita i warning (codice pulito = requisito di consegna)	tutti
-g	Simboli di debug (per gdb/valgrind)	debug

6.2 Vettorizzazione e diagnostica (M1)

Flag	Cosa fa
-ftree-vectorize	Abilita l'auto-vettorizzazione (gia' implicita in -O3) -> binario <code>plain_autovec</code>
-fno-tree-vectorize	Disabilita l'auto-vettorizzazione -> baseline scalare <code>plain_baseline</code>
-fopt-info-vec-optimized=file	Scriva nel file i loop che il compilatore HA vettorizzato
-fopt-info-vec-missed=file	Scriva i loop che ha tentato ma non vettorizzato (e perche')
-funroll-loops	Srotola i loop (piu' ILP, meno overhead di branch; non sempre conviene)
-fsanitize=thread	ThreadSanitizer: rileva data race a runtime (build di debug M2)
-DRUNTIME_SCHEDULE	Macro del progetto: compila <code>schedule(runtime)</code> -> <code>hashjoin_omp_runtime</code>

6.3 nvcc (CUDA, M1)

Flag	Cosa fa
-gencode arch=compute_80,code=sm_80	Genera codice per Ampere SM 8.0 (la GPU A30 di node09)

Flag	Cosa fa
-O3	Ottimizzazione del codice host/device
-std=c++17	Standard C++ per il codice CUDA

6.4 OpenMP - variabili d'ambiente (M3, M4 ibrido)

Variabile	Cosa fa
OMP_NUM_THREADS=n	Dimensione del team di thread
OMP_PROC_BIND=close	Pinna i thread su place consecutivi (riempie un socket prima di sconfinare) -> localita' NUMA
OMP_PROC_BIND=spread	Distribuisce i thread sui place (max banda, ma rompe la localita')
OMP_PROC_BIND=master false	Sui core del master / nessun binding
OMP_PLACES=cores	Un "place" = un core fisico (i thread si legano ai core)
OMP_PLACES=threads sockets	Place = HW thread (usa l'HT) / socket intero
OMP_SCHEDULE="dynamic,1"	Politica letta da <code>schedule(runtime)</code> (usata nello <code>schedule-sweep</code>)
OMP_DYNAMIC=false	Disabilita l'aggiustamento automatico del numero di thread (timing stabile)
OMP_MAX_ACTIVE_LEVELS=n	Livelli di parallelismo annidato consentiti

6.5 OpenMP - clausole principali (teoria L19/L22)

Clausola	Cosa fa
<code>schedule(static[,chunk])</code>	Iterazioni assegnate a priori in blocchi: overhead minimo, ideale per carichi regolari
<code>schedule(dynamic[,chunk])</code>	Iterazioni assegnate a domanda : bilancia carichi irregolari (skew); chunk piccolo = piu' fine
<code>schedule(guided[,chunk])</code>	Chunk decrescendenti : compromesso tra static e dynamic
<code>schedule(runtime)</code>	Politica decisa a runtime da <code>OMP_SCHEDULE</code>
<code>reduction(+:var)</code>	Ogni thread ha una copia privata, combinate alla fine: niente contesa, niente lock
<code>private / firstprivate / lastprivate(var)</code>	Variabile privata / inizializzata col valore esterno / con l'ultimo valore esportato
<code>shared(var)</code>	Variabile condivisa tra i thread
<code>default(none)</code>	Forza lo scoping esplicito di ogni variabile (buona pratica, evita bug)
<code>collapse(n)</code>	Fonde n loop annidati in un unico spazio di iterazione
<code>nowait</code>	Rimuove la barriera implicita a fine worksharing
<code>num_threads(n)</code>	(single/for) -> sovrappone le fasi
<code>if(expr)</code>	Numero di thread per quella regione parallela
	Esegue in parallelo solo se expr e' vera (altrimenti seriale)
<code>task / taskgroup / taskwait</code>	Crea un task / attende tutti i task del gruppo / attende i figli diretti
<code>depend(in out inout:var)</code>	Dipendenze dati tra task (ordina l'esecuzione)
<code>untied</code>	Il task puo' migrare di thread agli scheduling point (default: tied)
<code>final(expr) / priority(p)</code>	Task finale (non genera altri task) / suggerimento di priorita'
<code>critical / atomic / barrier</code>	Sezione mutuamente esclusiva / update atomico hardware / barriera esplicita

6.6 MPI - livelli di thread e lancio (L24-L26)

Elemento	Cosa fa
MPI_THREAD_SINGLE	Un solo thread nel processo (MPI puro)
MPI_THREAD_FUNNELED	Piu' thread, ma solo il main chiama MPI (usato dall'ibrido)
MPI_THREAD_SERIALIZED	Qualsiasi thread chiama MPI, ma non concorrentemente
MPI_THREAD_MULTIPLE	Qualsiasi thread, anche concorrentemente (lock interni -> overhead)
mpirun -n R	Lancia R rank (run locale, senza SLURM)
srun --mpi=pmix	Bootstrap dei rank via PMIx (necessario sotto SLURM con Open MPI 5)
module load openmpi5	Non esiste su questo cluster (Lmod: unknown module): mpicxx/mpirun di Open MPI 5.0.3 sono gia' nel PATH di default
--bind-to core / --map-by node	(Open MPI) binding dei rank ai core / mappatura per nodo

6.7 SLURM - flag di allocazione

Flag	Cosa fa
--nodes=N	Numero di nodi fisici
--ntasks=R	Numero totale di task/processi (= rank MPI)
--ntasks-per-node=k	Task per nodo (32 per MPI puro, 1 per ibrido)
--cpus-per-task=T	Core riservati per task (serve a OpenMP: T thread)
--exclusive	Nodo riservato in esclusiva -> timing pulito, niente vicini rumorosi
--partition=normal	Coda/partizione (gpu-excl per node09/GPU)
--nodelist=node09	Forza un nodo specifico (M1 -> node09)
--time=HH:MM:SS	Limite di wall-clock (oltre, il job viene ucciso)
--gres=gpu:1	Richiede 1 GPU (se servisse esplicitarlo per la A30)
--hint=nomultithread	Usa solo i core fisici, ignora l'Hyper-Threading

6.8 Affinita' e misura (L18) - utili per spiegare/diagnosticare

Comando	Cosa fa
lscpu	Topologia: socket, core/socket, thread/core, cache, nodi NUMA
numactl --hardware	Nodi NUMA, memoria per nodo, matrice delle distanze
numactl --cpunodebind=0 --membind=0 ./bin	Pinna esecuzione e memoria al nodo NUMA 0 (test localita')
taskset -c 0-15 ./bin	Pinna il processo ai core 0-15 (un socket)
perf stat -e cache-misses,LLC-load-misses ./bin	Contatori HW: cache miss, traffico LLC (prova del memory-bound)
perf stat -e mem_load_uops... / numastat -p <pid>	Accessi remoti NUMA (per dimostrare il dip a p=20, non solo ipotizzarlo)

Da dire all'orale: i flag non sono decorazioni. `-march=native` vs `-march=ivybridge` e' una scelta di **correttezza** (SIGILL); `OMP_PROC_BIND=close+OMP_PLACES=cores` e' cio' che fa funzionare il **first-touch NUMA**; `schedule(dynamic,1)` vs `static` e' la differenza tra subire o assorbire lo **skew**; `MPI_THREAD_FUNNELED` invece di `MULTIPLE` evita **lock inutili**. Ogni flag ha una motivazione dalla teoria.